

Proyecto final

Seminario de Rust 2026

Bot de Discord para jugar el truco

Integrantes:

Ezequiel

Mateo

Vazquez

Introducción

El presente proyecto consiste en el desarrollo de un bot de Discord programado en Rust que actúa como árbitro automatizado para partidas de Truco Argentino 1vs1 por turnos. Por el cual deberá gestionar el flujo completo del juego, validando la legitimidad de las jugadas, los cantos y la asignación de puntajes sin alterar el estado de la partida ante acciones inválidas

Objetivos

Aplicar las herramientas y paradigmas avanzados del lenguaje Rust para construir un motor de juego multijugador concurrente.

Como a su vez provee al Bot de una arquitectura aislada de la lógica funcional de las reglas del juego y la capa de comunicaciones de Discord. De modo que se garantice la concurrencia del sistema entre partidas que se realicen.

Fundamentos y cuestiones técnicas:

Principalmente para el desarrollo del proyecto nos orientamos en una estrategia de diseño orientado al dominio, por el cual inicialmente nos encargamos de construir y validar el "cerebro" o motor del juego de forma totalmente aislada. De modo que el bot debe ser capaz de actuar como un árbitro autónomo que reciba intenciones de juego, aplique las reglas básicas del Truco, valide turnos y gestione el puntaje en un entorno local, sin depender de librerías de red ni de la API de Discord.

Este aislamiento tiene como finalidad que las reglas de negocio no queden alteradas por la infraestructura. De manera que si la plataforma de comunicación cambiará en el futuro, la lógica central del árbitro permanezca intacta y completamente operacional para las próximas partidas.

Cuestiones básicas:

1. ¿Por qué trabajar en Rust?

Rust es un lenguaje de programación de sistemas centrado en la seguridad, la concurrencia y el alto rendimiento. Tiene un sistema de memoria basado en el manejo de propiedad (ownership), préstamos (borrowing) y tiempos de vida (lifetimes), lo que permite garantizar la seguridad de memoria (memory safety) en tiempo de compilación sin recurrir a un recolector de basura (garbage collector). Esta arquitectura elimina de origen errores comunes como punteros nulos, accesos fuera de rango o referencias colgadas, e impone un tratamiento explícito de los errores mediante tipos como Result y Option.

En cuanto a la concurrencia, Rust aplica el principio de concurrencia sin carreras de datos (fearless concurrency). El propio sistema de tipos y las reglas de préstamo garantizan que múltiples hilos no puedan mutar simultáneamente la misma posición de memoria, previniendo condiciones de carrera en tiempo de compilación.

Por último, su nivel de eficiencia radica en que se compila directamente a código de máquina nativo altamente optimizado a través de LLVM. Al utilizar abstracciones de costo cero (zero-cost abstractions), Rust ofrece un rendimiento y un control de recursos semejantes a C y C++.

2. ¿Por qué utilizar Discord como medio principal?

3. ¿Qué es la concurrencia?

4. ¿Qué es Tokio?

5. ¿Qué es Async?

Cuestiones técnicas:

6. ¿Qué responsabilidad cumplen `#[tokio::main]`, `async fn` y `#[poise::command]` en el ciclo de una interacción?

7. ¿Por qué el estado compartido necesita Arc y Mutex en el ejemplo, y qué protege el lock?

8. ¿Por qué es incorrecto mantener un MutexGuard vivo mientras se ejecuta `ctx.reply(...).await`?

9. ¿Cómo distingue Poise una respuesta pública de una efímera y por qué “/truco mano” debe usar la segunda?

10. ¿Por qué la clave correcta del repositorio de juego es el canal y no el usuario?

11. ¿Qué tipos pertenecen al adaptador Discord y cuáles al dominio puro?

Diseño del dominio y Modelo de Datos

Flujo funcional y ciclo de vida

Mostrar las capas/etapas de cómo interactúan código y discord para hacer funcionar el juego . Deberá incluir diagramas o esquemas

Fuentes de interés

Herramientas